SMART DAIRY

AI-Powered Dairy Management Platform

Software Name:	Smart Dairy Platform
Version:	1.0.0
Authors:	Smart Dairy Development Team
Owner:	Smart Dairy Development Team
Date:	October 22, 2025

ABSTRACT

Purpose of the Software:

Smart Dairy revolutionizes dairy farming through AI-powered health monitoring, disease detection, feed optimization, and production analytics.

Technologies Built From:

Python Django 5.2.7, Google Gemini 2.0 Flash AI, Bootstrap 5.3.0, SQLite/PostgreSQL

Platform:

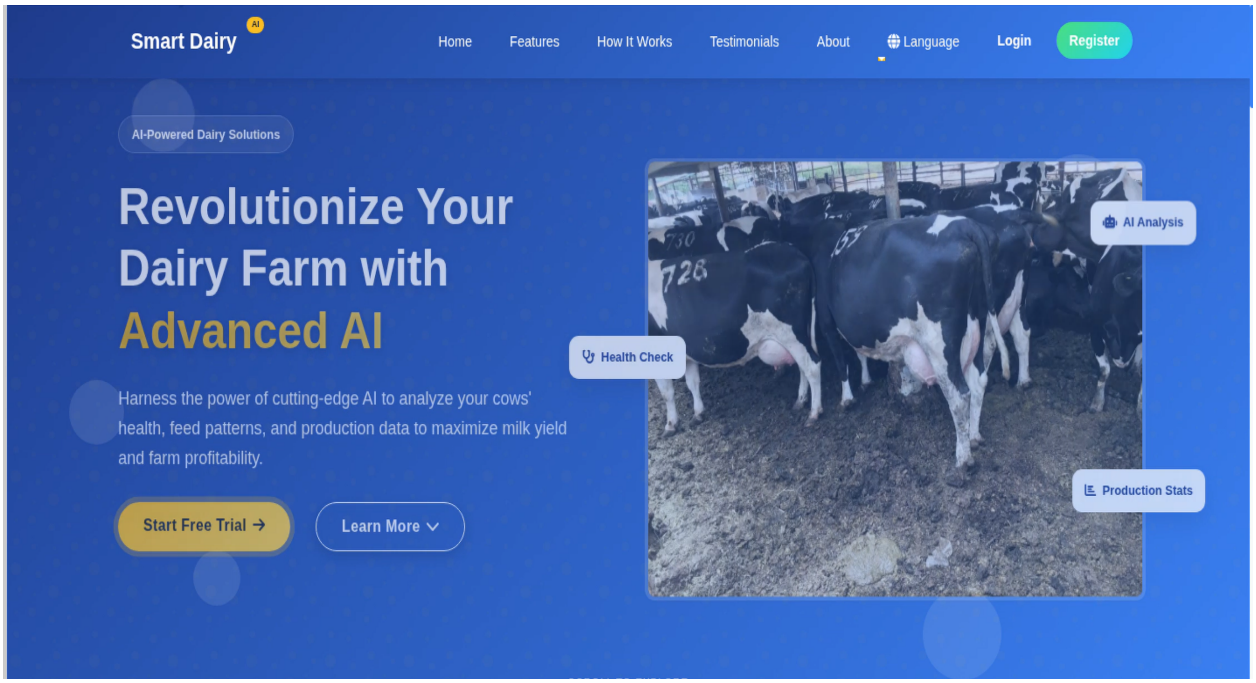
Web-based application deployable on cloud platforms (AWS, Google Cloud, Azure)

INTRODUCTION

Smart Dairy is an innovative web-based platform that leverages advanced artificial intelligence to transform traditional dairy farming practices in Kenya and East Africa. The software addresses critical challenges faced by dairy farmers including early disease detection, optimal feed management, and production optimization. Key Functionalities: • AI-powered disease detection through computer vision analysis of cow photographs • Personalized feed recommendations based on individual cow characteristics (breed, age, weight, health status) • Bilingual AI assistant providing expert advice in English and Swahili languages • Real-time health monitoring and production analytics dashboard • Comprehensive cow management system with detailed record keeping • Secure farmer registration and authentication system • Mobile-responsive interface for field use How the Software Works: Farmers begin by registering on the platform and creating their farm profile. They can then add individual cows to their digital herd, recording essential details like breed, age, weight, and health status. The AI system analyzes uploaded cow photos to detect diseases early, providing immediate diagnostic insights and treatment recommendations with urgency levels. The feed optimization module generates personalized nutrition plans considering each cow's unique characteristics and production goals. Farmers can interact with the bilingual AI assistant for instant expert advice on various dairy farming topics. The system continuously learns from data to improve recommendations and predictions. The platform's dashboard provides comprehensive analytics on herd health, production trends, and feed efficiency, enabling data-driven decision making for improved farm profitability and cow welfare.

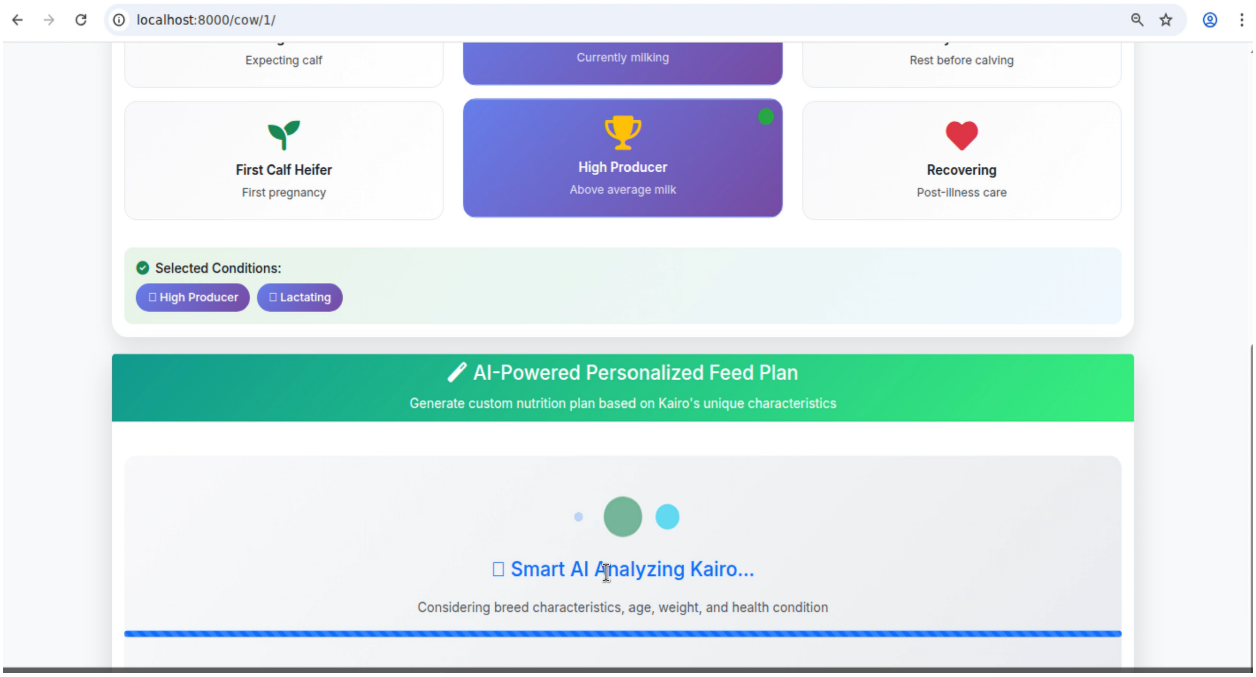
SOFTWARE DOCUMENTATION

4.1 Main Landing Interface



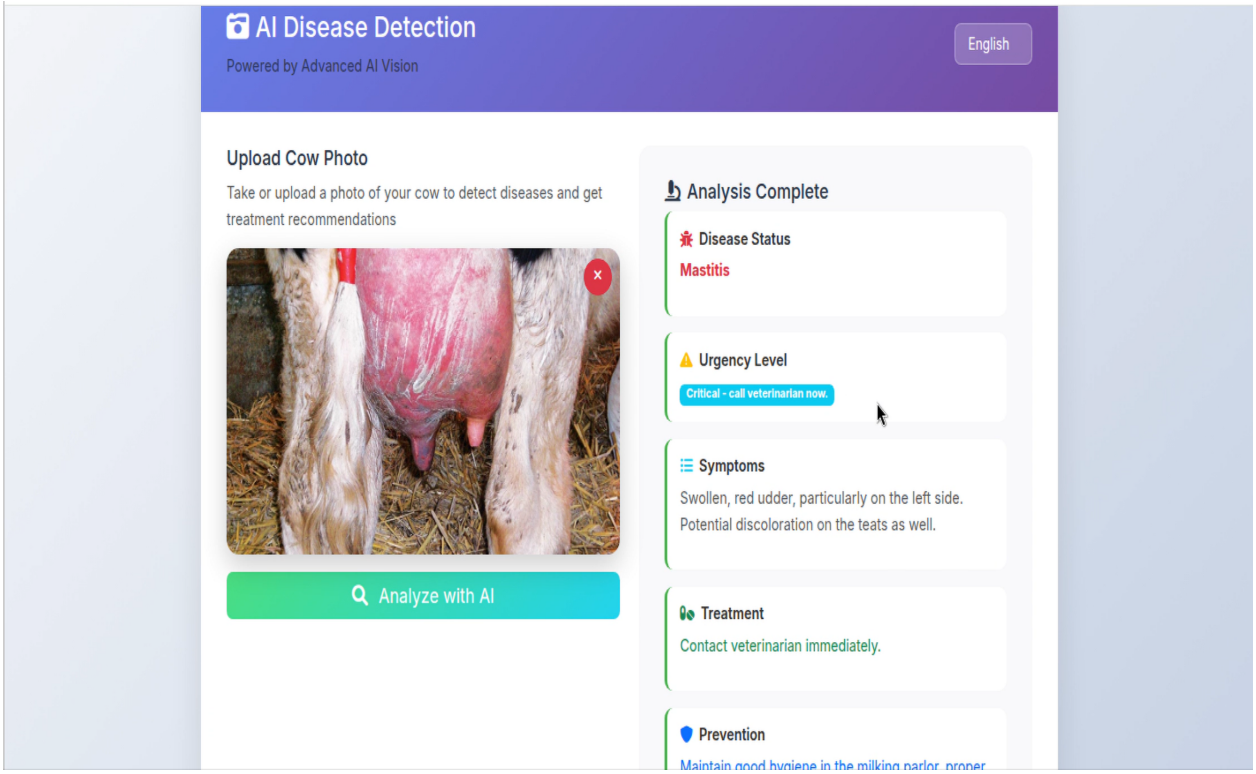
The landing page showcases AI-powered dairy solutions with comprehensive navigation.

4.2 AI Feed Personalization



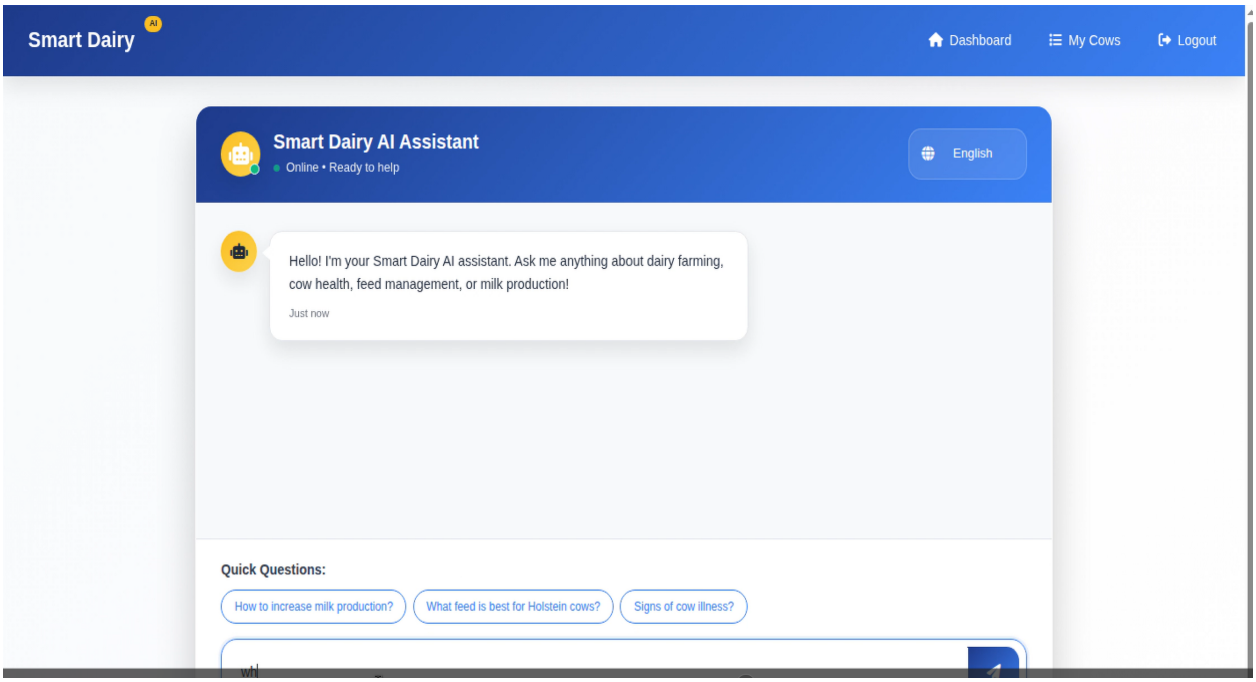
AI generates personalized nutrition plans based on cow characteristics.

4.3 Disease Detection System



Computer vision provides instant disease diagnosis with treatment recommendations.

4.4 Bilingual AI Assistant



Natural language processing supports English and Swahili.

4.5 Core Application Code

```
# Database Models class Cow(models.Model): owner = models.ForeignKey(User, on_delete=models.CASCADE)
name = models.CharField(max_length=100) breed = models.CharField(max_length=100) age =
models.IntegerField() weight = models.FloatField() health_condition = models.CharField(max_length=20)
class DailyFeedRecord(models.Model): cow = models.ForeignKey(Cow, on_delete=models.CASCADE) date =
models.DateField() protein_feed = models.FloatField() silage = models.FloatField() minerals =
models.FloatField() is_ai_recommended = models.BooleanField(default=False)

# AI Service Implementation class AIService: def __init__(self):
genai.configure(api_key=settings.AI_API_KEY) self.model =
genai.GenerativeModel('gemini-2.0-flash-exp') def analyze_disease_photo(self, image_file,
language='en'): image = Image.open(image_file) prompt = self._build_disease_analysis_prompt(language)
response = self.vision_model.generate_content([prompt, image]) return
self._parse_disease_response(response.text, language) def get_feed_recommendation(self, cow_data,
language='en'): prompt = self._build_feed_prompt(cow_data, language) response =
self.model.generate_content(prompt) return self._parse_feed_response(response.text, language)
```

```
# Django Views @login_required def dashboard(request): user_cows =
Cow.objects.filter(owner=request.user) context = {'total_cows': user_cows.count(), 'cows':
user_cows[:5]} return render(request, 'main/dashboard.html', context) def
ai_disease_detection(request): if request.method == 'POST' and request.FILES.get('cow_image'): image
= request.FILES['cow_image'] ai_service = AIService() analysis =
ai_service.analyze_disease_photo(image) return JsonResponse(analysis) return render(request,
'main/disease_detection.html')
```